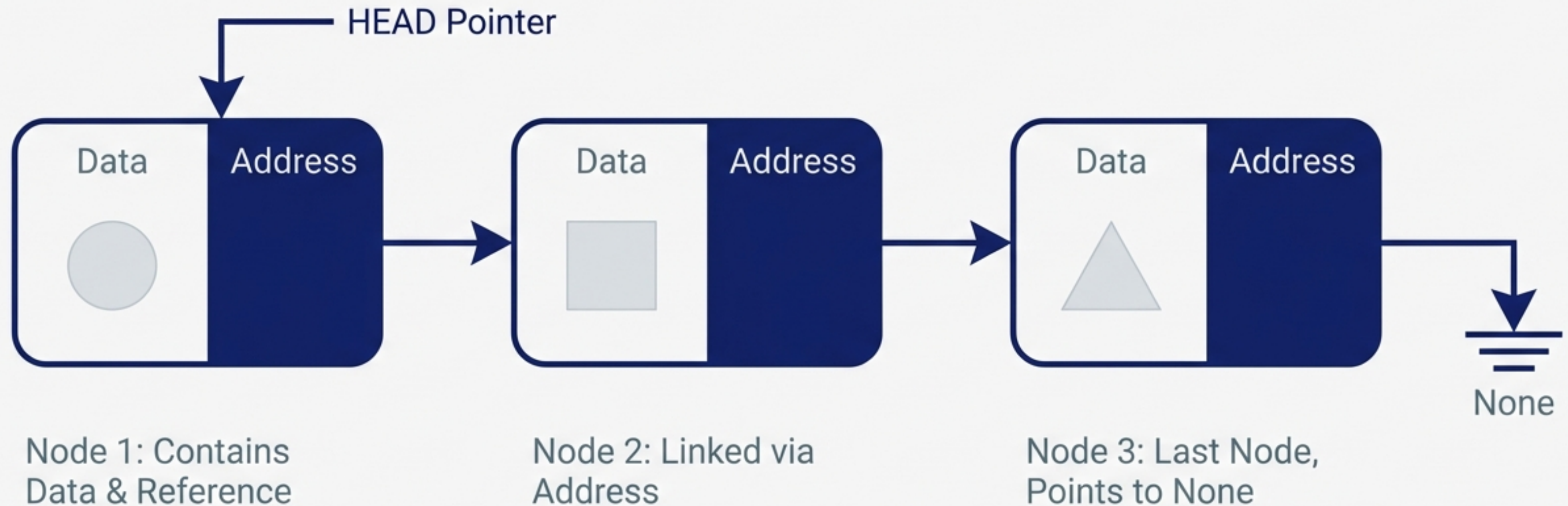


Mastering Singly Linked Lists in Python

From Memory Concepts to Interview-Ready Code

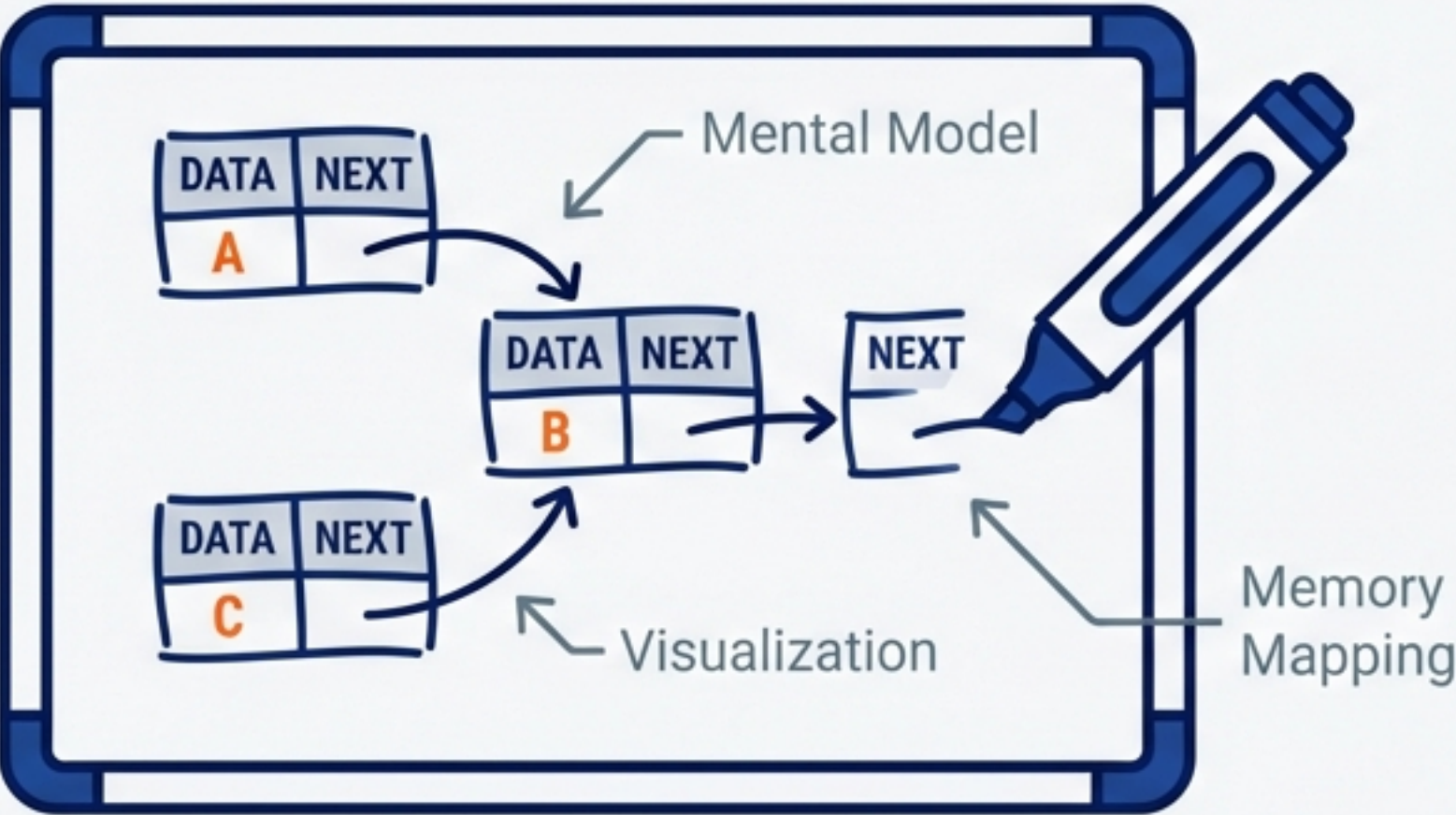


Methodology: Logic First, Syntax Second | Based on Chai aur Code

Why Interviewers Love Linked Lists

The Logic Test

Interviews rarely start with code. They start with drawings. Interviewers assess your ability to visualize memory movement before checking your Python syntax.



Under the Hood



Spotify Queue
Play Next functionality dynamically links songs.



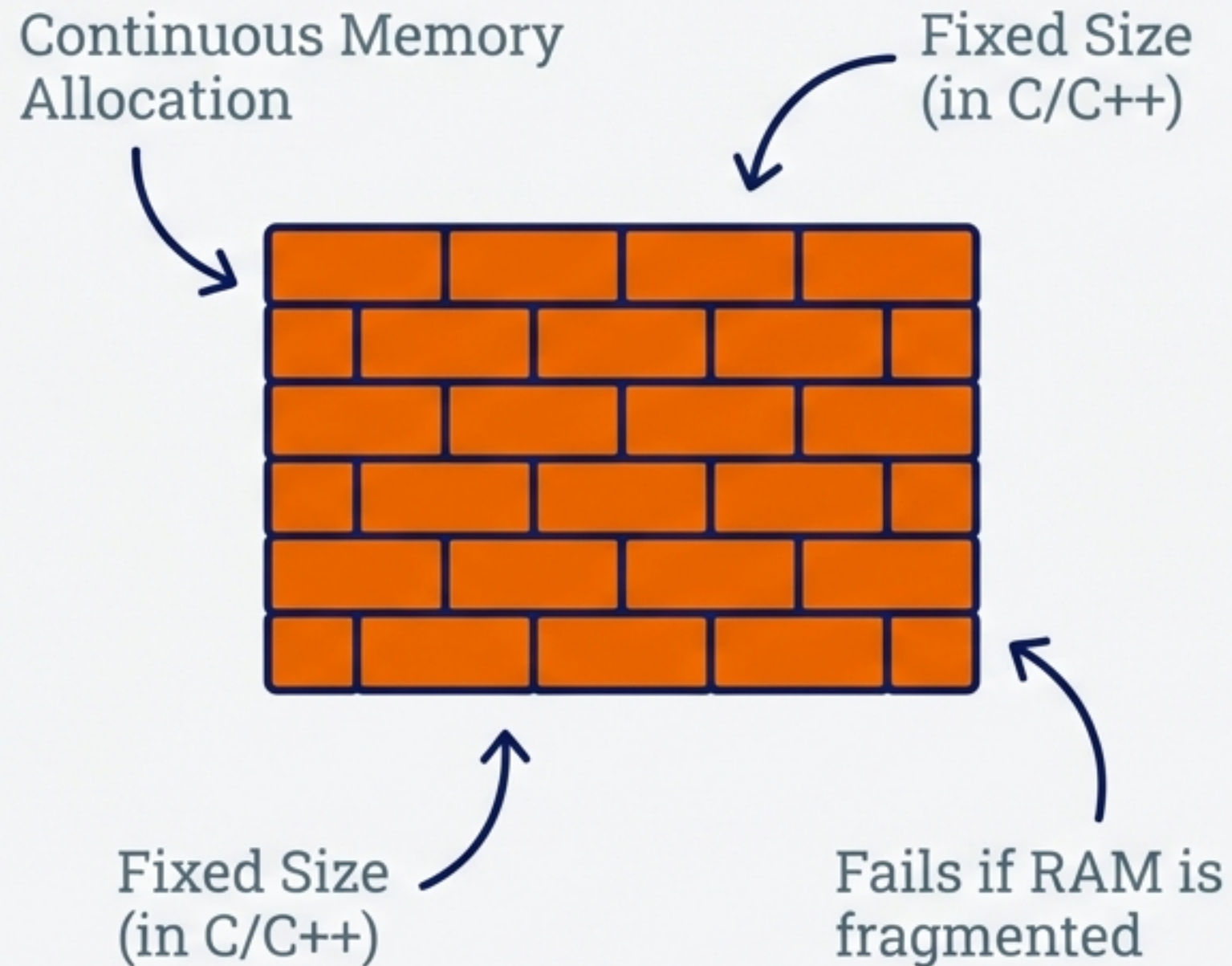
Browser Navigation
Back/Forward buttons rely on linked history.

Foundation
Prerequisite for Graphs & HashMaps.

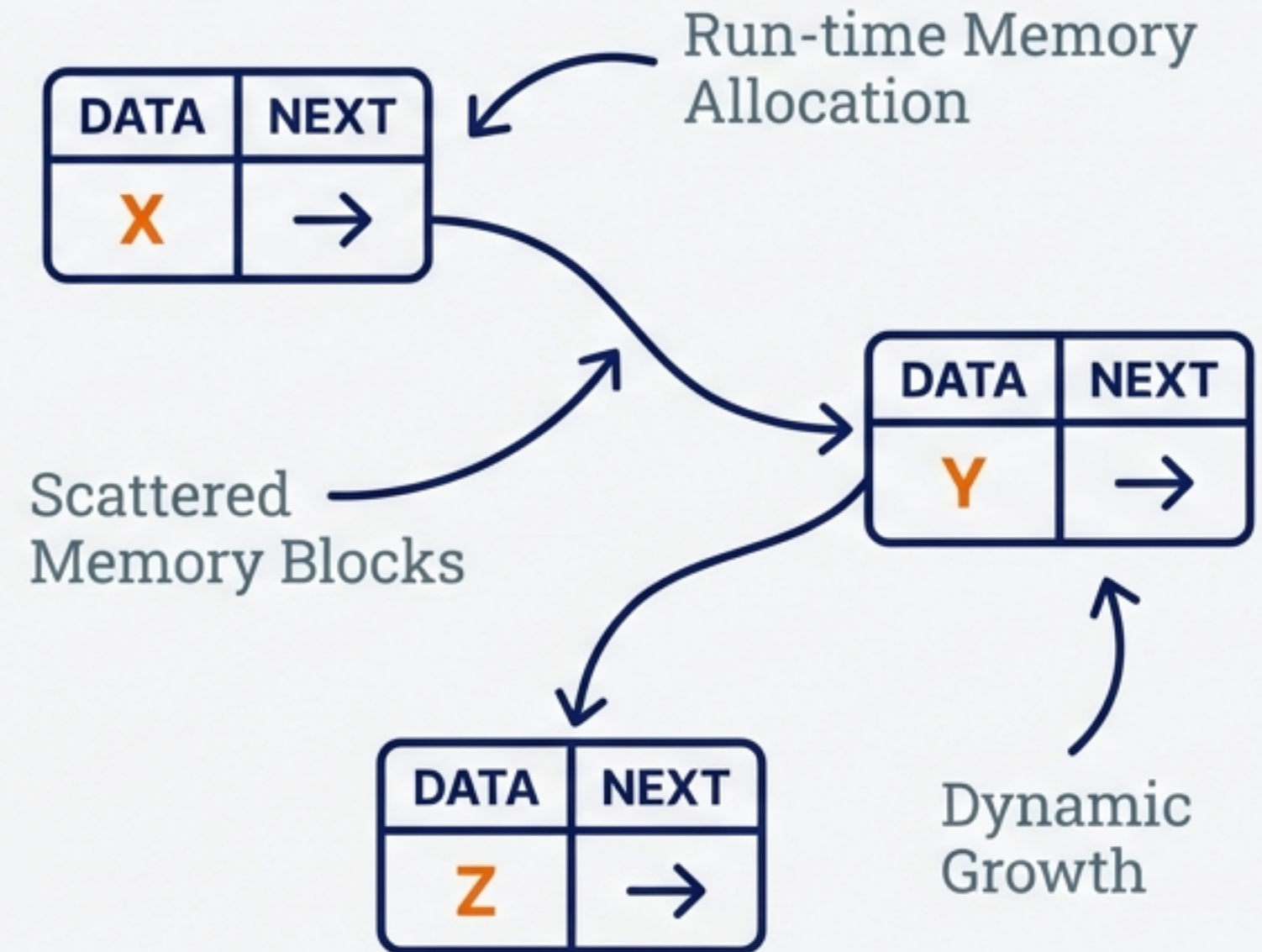


The Problem with Arrays vs. The Flexibility of Lists

Arrays: The Brick Wall



Linked Lists: The Archipelago



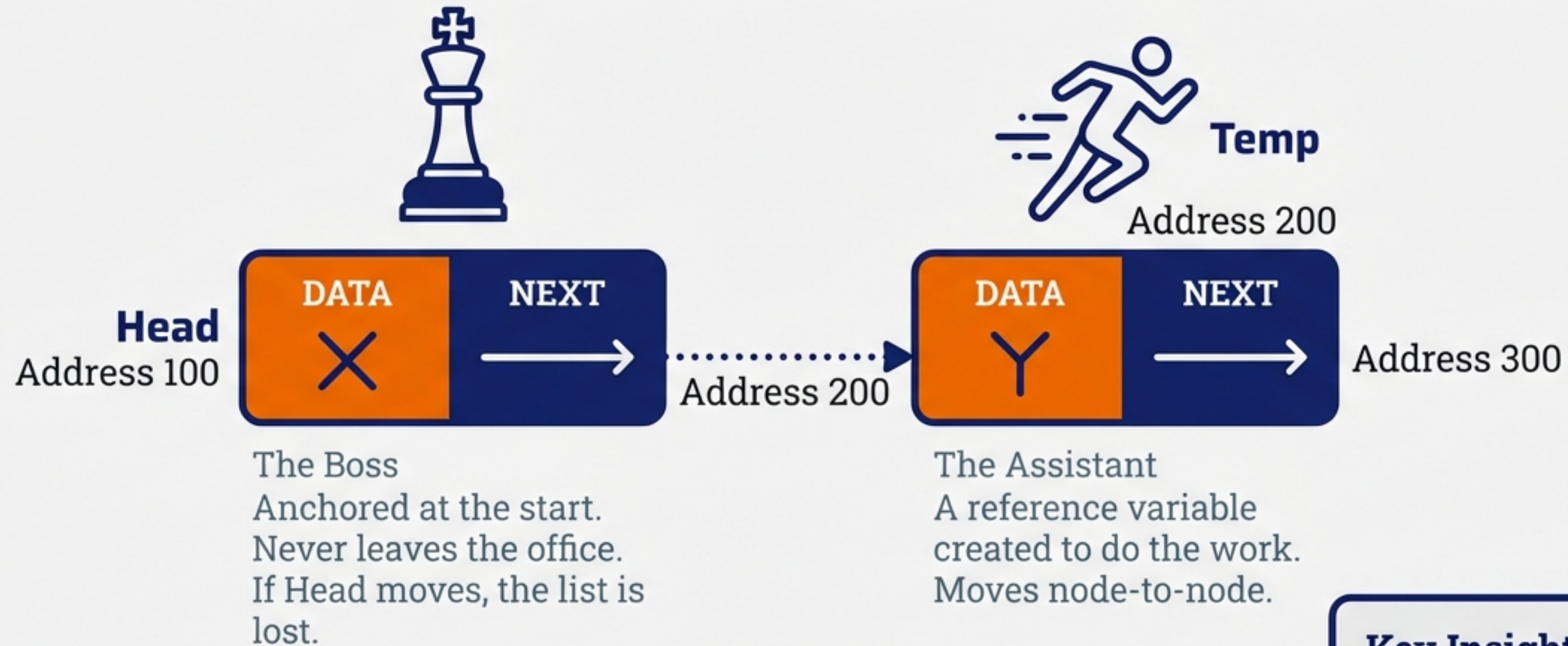
Anatomy of a Node

DATA	NEXT
10 Holds the Information (Int, String, Object)	Address: 200 Reference to the next node

```
class Node:
    def __init__(self, data, next_node=None):
        self.data = data      # The Information
        self.next = next_node # The Reference
```

Note: Python uses Reference Variables, not raw memory pointers.

The Golden Rule of Traversal

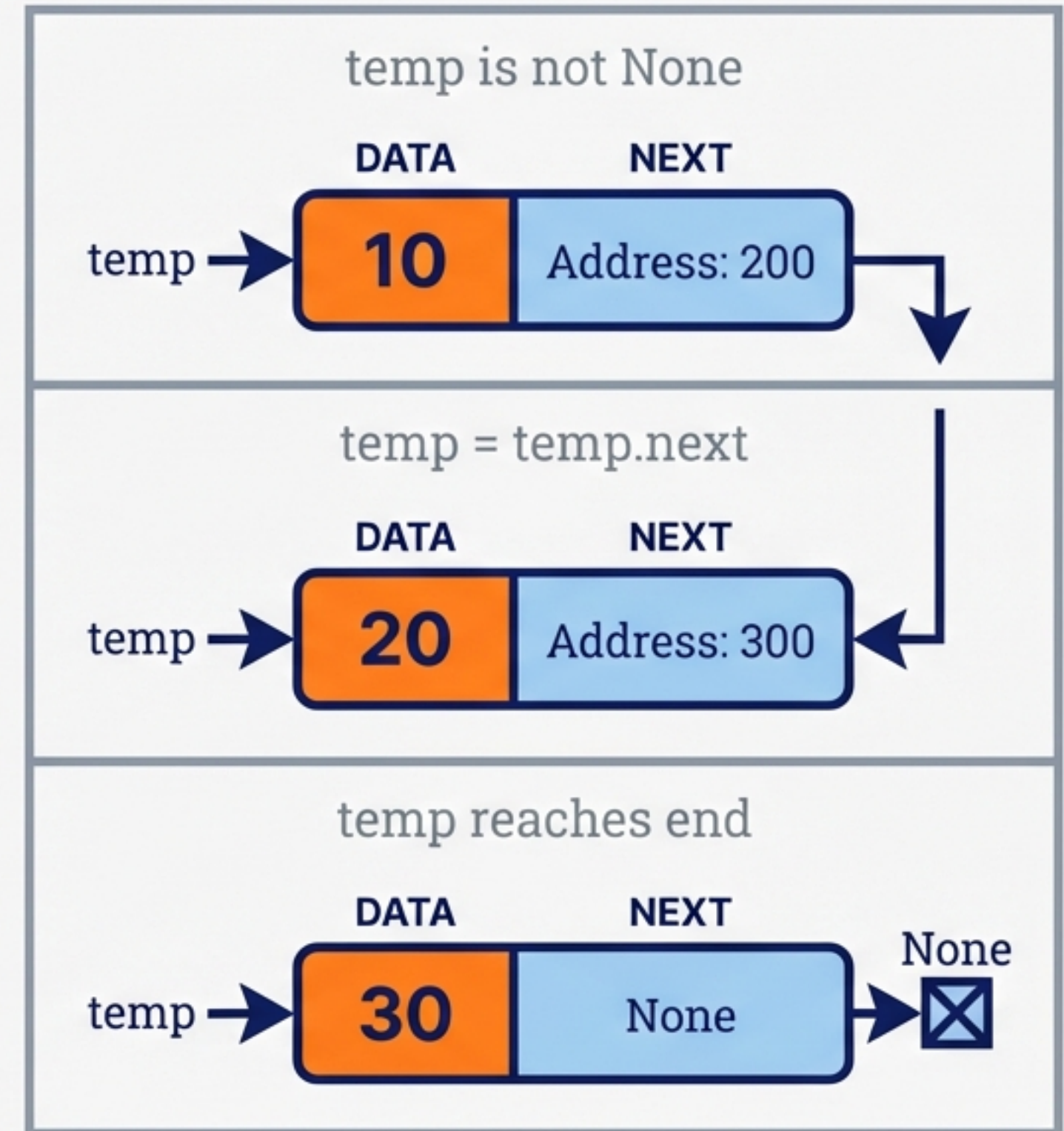


Key Insight Box

Always use a temporary variable (temp) for traversal.
Never iterate with self.head.

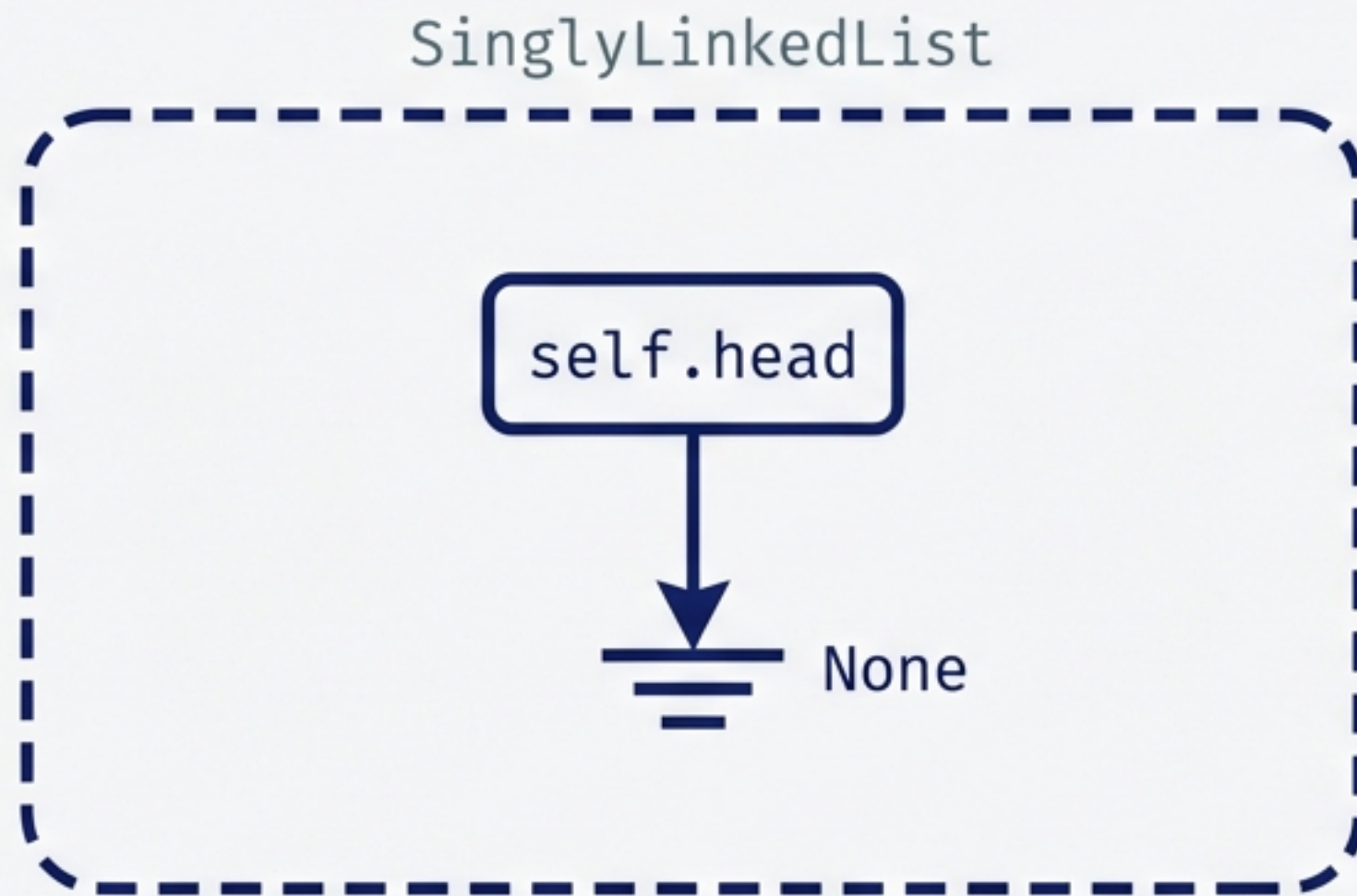
The Core Loop: Walking the List

```
def print_linked_list(self):  
    temp = self.head  
    while temp is not None:  
        print(temp.data)  
        temp = temp.next # The Step Forward
```



The Singly Linked List Class

We encapsulate the logic in a wrapper class. This manages the `head` pointer.



Initial State: An empty wrapper waiting for nodes.

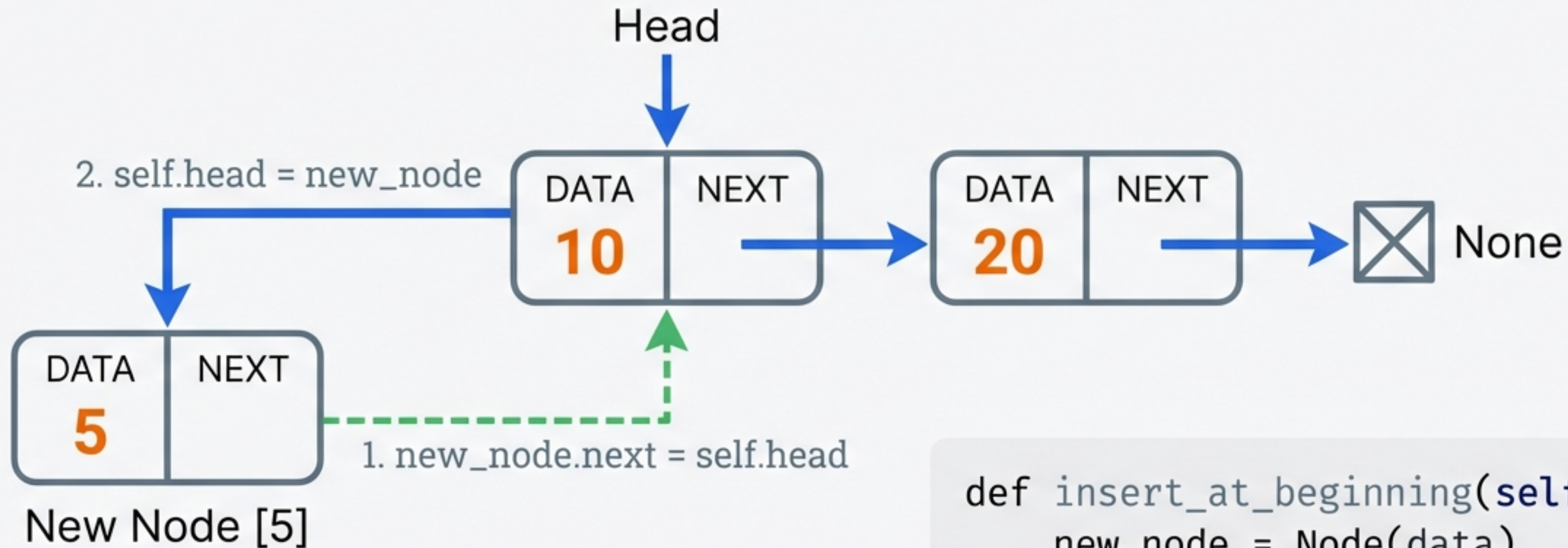
```
class SinglyLinkedList:
    def __init__(self):
        self.head = None # Initially empty
```


Operation: Insert at End (Append)



```
def insert_at_end(self, data):  
    new_node = Node(data)  
    if self.head is None:  
        self.head = new_node  
        return  
    temp = self.head  
    while temp.next is not None: # Walk to end  
        temp = temp.next  
    temp.next = new_node # Link the new node
```

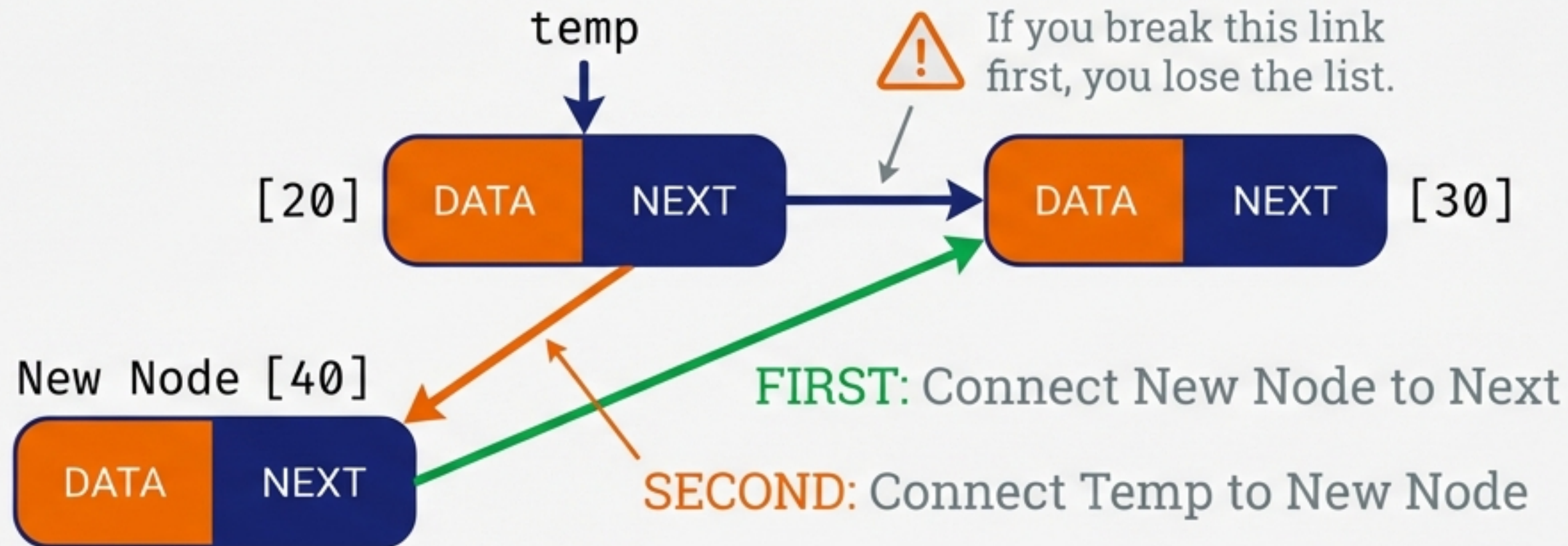

Operation: Insert at Beginning (Prepend)



```
def insert_at_beginning(self, data):  
    new_node = Node(data)  
    new_node.next = self.head  
    self.head = new_node
```


Operation: Insert in Middle

The Order of Operations is Critical

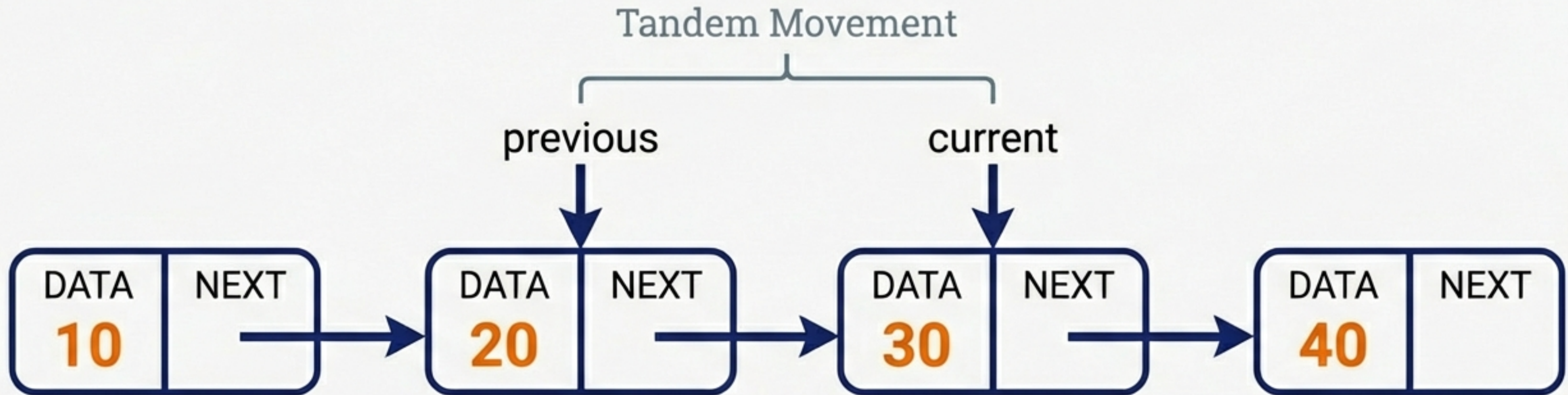


```
# ... traversal logic ...  
new_node.next = temp.next # Step 1  
temp.next = new_node     # Step 2
```


Deletion Logic: The Two-Pointer Approach

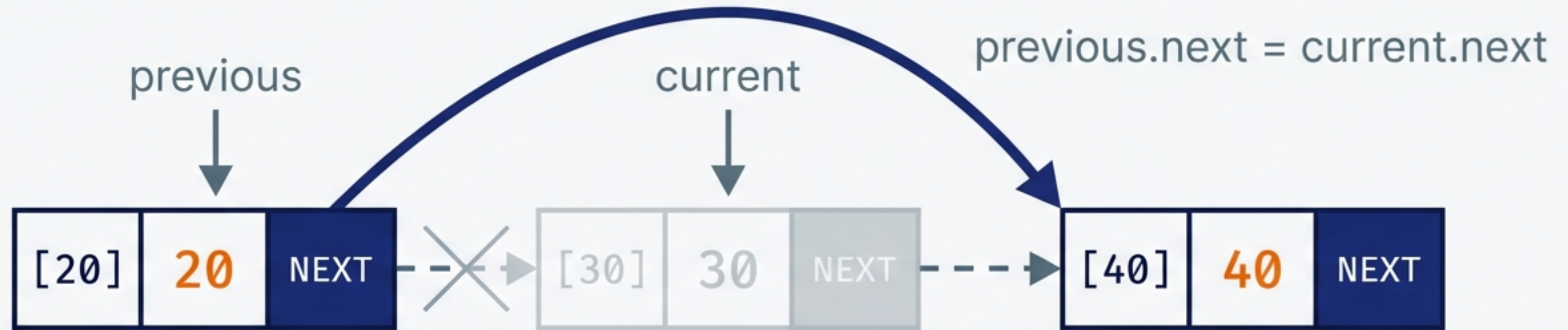
To delete Node B, Node A must connect to Node C.

But from Node B, we cannot see Node A.



We keep a 'Previous' pointer one step behind to bridge the gap.

Implementing Deletion (The Snipping Tool)



```
while current is not None:
    if current.data == value:
        previous.next = current.next # Snip the link
        break
    previous = current # Move Prev forward
    current = current.next # Move Curr forward
```


Handling the Edge Cases

Case 1: Deleting the Head

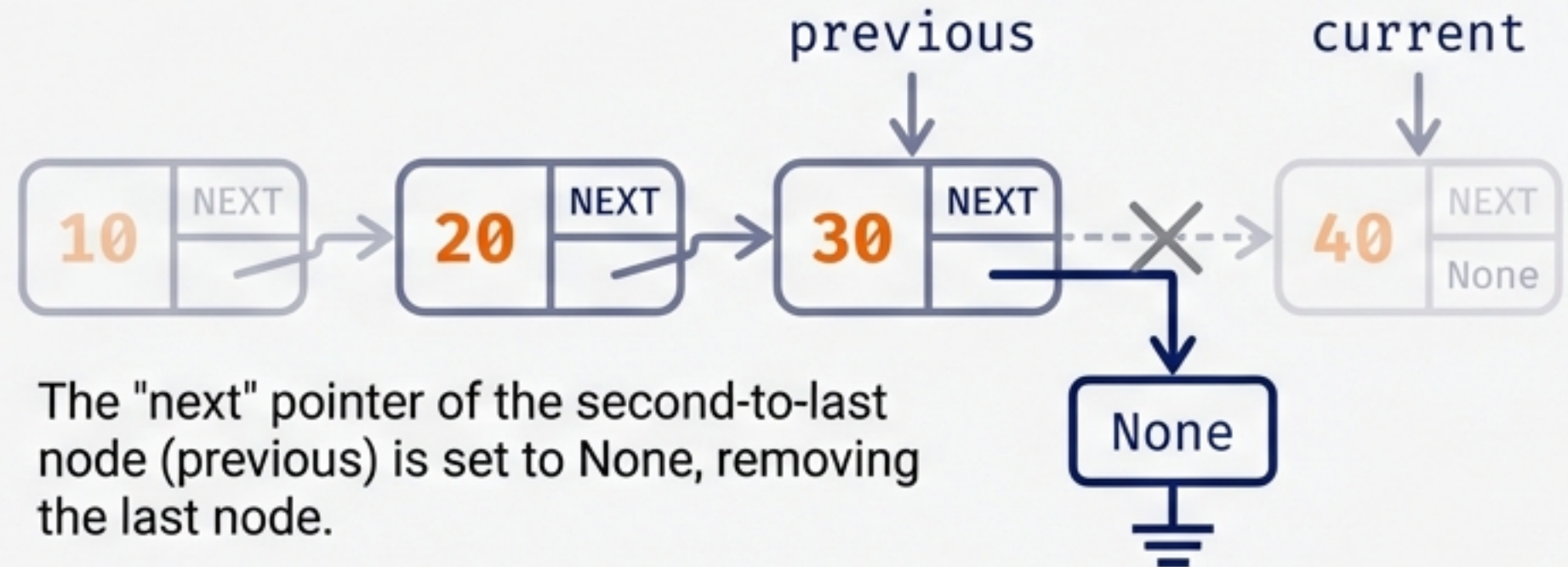
```
if self.head.data == value:  
    self.head = self.head.next
```



The head pointer is updated to the next node, effectively dropping the first node from the list.

Case 2: Deleting the Last Node

```
if current.next is None:  
    previous.next = None
```



The "next" pointer of the second-to-last node (previous) is set to None, removing the last node.

The Full Picture: Pros & Cons



PROS

- **Dynamic Size:** Run-time allocation. No wasted memory.
- **Efficient Insertion/Deletion:** $O(1)$ if pointer is known (no shifting elements like arrays).



CONS

- **No Reverse Traversal:** Strictly forward direction.
- **Sequential Access:** Search is $O(n)$. Cannot jump to index [5].
- **Memory Overhead:** Extra memory needed for the pointer in every node.

Your Next Steps

“Code logic first, syntax second. If you can draw it, you can code it.”

- ▣ Practice Sheets: Visit [Masterji.co](https://masterji.co)
- ▣ Community Challenge: Convert this logic to C++ or Java in the comments.

Coming Soon: Doubly Linked Lists (Moving Backward!)